Embedded Data Structure
(Lecture Note 11)

# Queue

2025. 4. 8

Prof. Wonjun Kim

School of Electrical and Electronics Engineering

# Review of Last Class (1/3)

KU KONKUK UNIVERSITY

- **Stack by array and linked-list representations**
  - **FILO (first-in / last-out) memory structure**

    * Two main actions : **push** and **pop**

    * Remember the meaning of those actions in linked lists !

  - **Application of stack : CALC (postfix equation)**

    * What is the main advantage of postfix representation ?

---

※ Example : Convert (1+(2*3)) to the postfix notation

**Pseudo code (for example) :**

1. Ignore "(" and go further
2. Print the operand "1"
3. Push the operator "+"
4. When you meet ")", pop and print out

What is the result ?

# Review of Last Class (2/3)

- **Now, we are ready to implement it !**
  - **Input (src) : string of equation / output (dst) : postfix representation**

```
void postfix(char *dst, char *src)
{
    char c;
    init_stack();

    while(*src) {
        if(*src == ')')
        {
            *dst++ = pop();
            *dst++ = ' ';
            src++; // what does it mean ?
        }
        else if (*src == '+' || *src == '-' ||
                 *src == '*' || *src == '/')
        {
            [redacted]
            src++;
        }
        else if (*src >= '0' && *src <= '9')
        {
            do{
                [redacted]
            }while(*src >= '0' && *src <= '9');
            *dst++ = ' ';
        }
        else
            src++;
    }  // end while
    *dst = 0;
}
```

# Review of Last Class (3/3)

- **Now, we are ready to implement it !**
  - **Main function for execution**

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

void main(){
    char exp[256];
    char src[256] = "(1+(2*3))";

    postfix(exp, src);
    printf("Postfix representation : %s₩n", exp);
}
```

Functions for stack are required !

(i.e., push and pop functions)

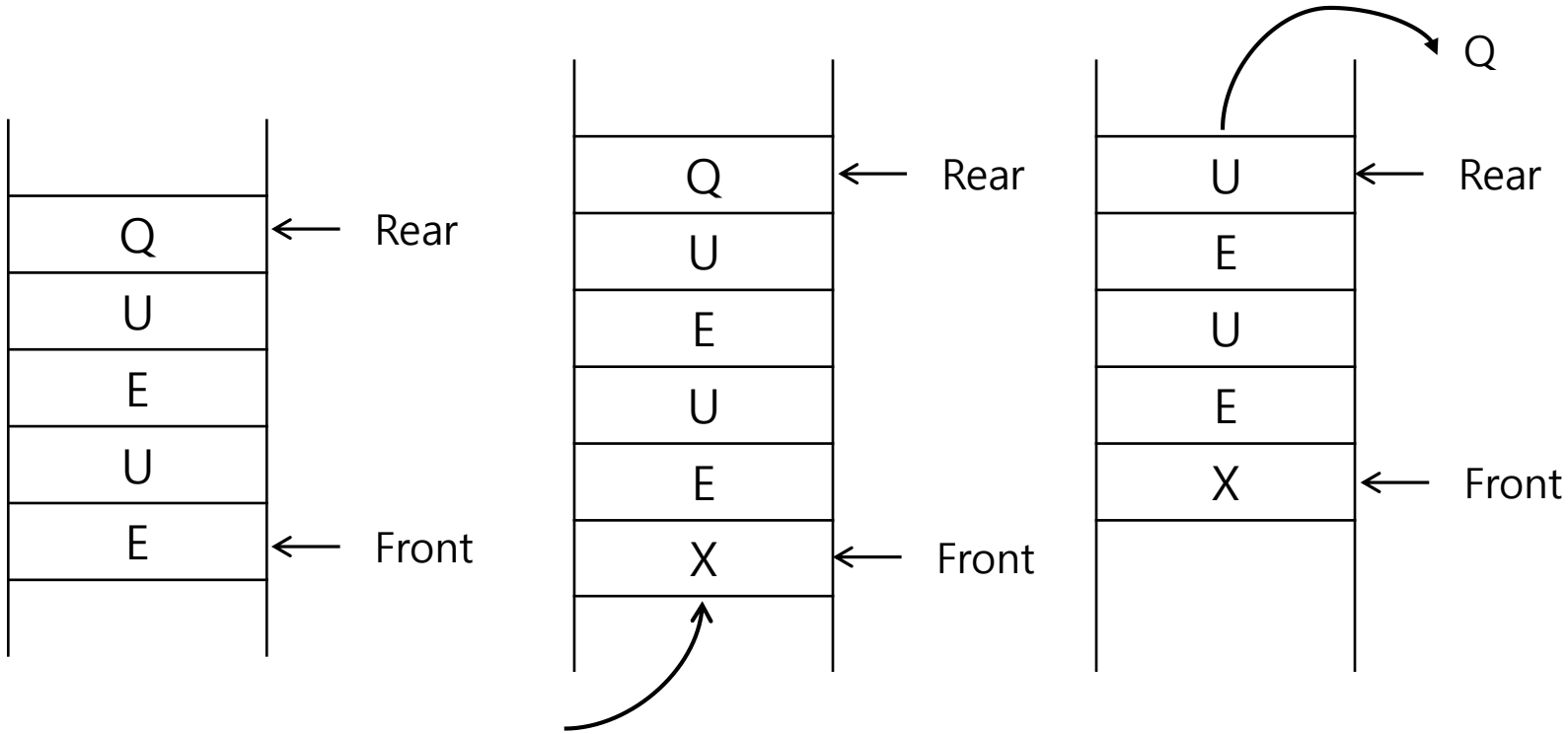
➡ **Check out the result !**

**※ Think about the computation of a given equation as well !**

## Basic concept of queue (FIFO)

- **Two main functions : "put" and "get"**

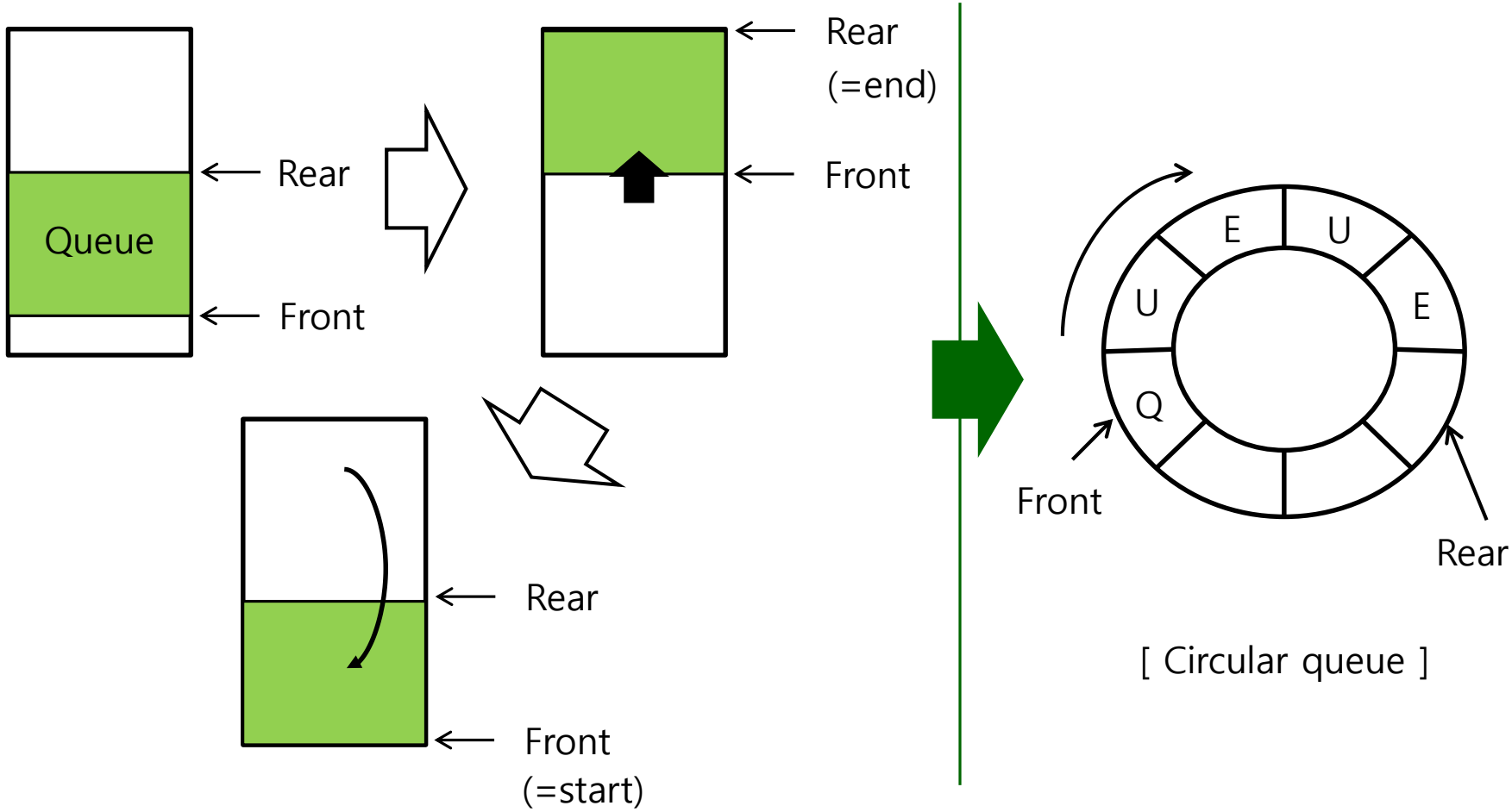
  - We need to track the position of "**rear**" and "**front**" to manage queue !




[ Queue ] [ Queue (put) ] [ Stack (pop) ]

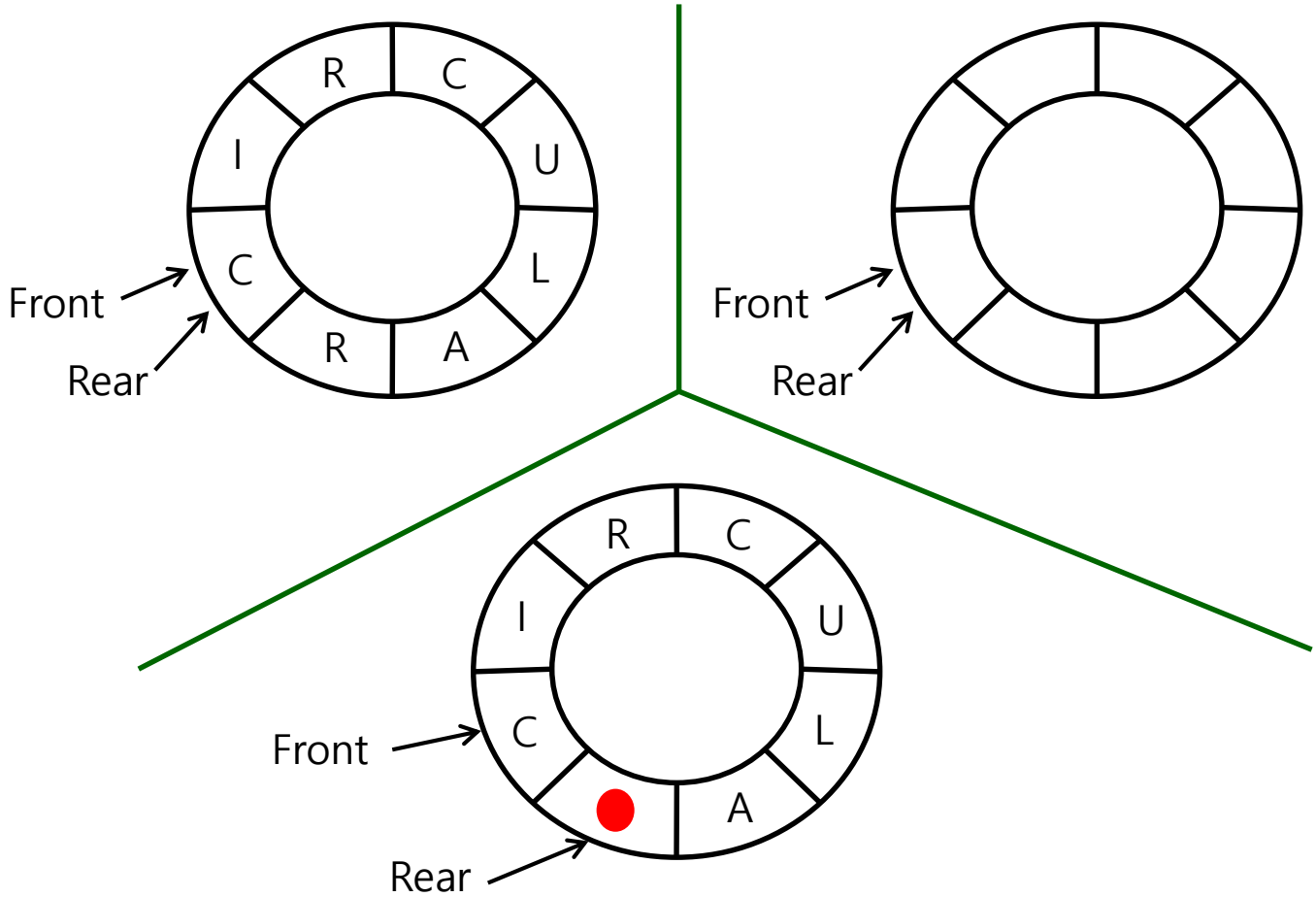
# Array-based Queue (1/4)

- **Control of the index for handling the array**
  - **Think about put→get→put→get→ ....**



# Array-based Queue (2/4)

- **Control of the index for handling the array – cont'd**
  - **Circular queue requires an empty space**




```
void init_queue()
{
    front = 0;
    rear = 0;
}

void clear_queue()
{
    front = rear;
}
```

# Array-based Queue (3/4)

- **Put and get**
  - **Easy to implement !**

```
int put(int k)
{
    if((rear+1)%MAX == front)
    {
        printf("Queue overflow !!!₩n");
        return -1;
    }

    rear = [                    ]

    return k;
}
```

➔ **Be careful to handle index !**

```
int get()
{
    int j;

    if(front == rear)
    {
        printf("Queue underflow !!!₩n");
        return -1;
    }

    j = queue[front];
    front = ++front%MAX;

    return j;
}
```

# Array-based Queue (4/4)

KU KONKUK UNIVERSITY

## ▪ Implementation of array-based queue

- **Easy to implement !**

```
#define MAX 10

int queue[MAX];
int front, rear;

void main()
{
    int k;
    init_queue();

    Make your code

}
```

**To do list :**

1. Put 3, 6, 9, 1, 6, 3 and print front→rear
2. Conduct get and print front→rear
3. Put 4, 8, 7, 2, 0 (what happens ?)
4. Initialize your queue
5. Conduct get (what happens ?)

```
void print_queue()
{
    int k;
    for(k = front; k != rear; k = ++k%MAX)
        printf("%-6d", queue[k];
}
```

# Linked list-based Queue (1/4)

- **Doubly linked list for front and rear index**
  - **Size of queue can be changed (reduction and increase)**
    - Put to the rear / get from the front ➔ **doubly linked list**

*** Structure definition**

```
typedef struct _dnode
{
    int key;
    struct _dnode *prev;
    struct _dnode *next;
}dnode;

(globally defined)
dnode *head, *tail;
```

*** Queue initialization**

```
void init_queue()
{
    head = (dnode *)calloc(1, sizeof(dnode));
    tail = (dnode *)calloc(1, sizeof(dnode));
    head->prev = head;
    head->next = tail;
    tail_prev = head;
    tail->next = tail;
}
```

# Linked list-based Queue (2/4)

- **Put and get**
  - **Link operation (insert and delete)**

```
int put(int k)
{
    dnode *t;
    if((t = (dnode*)malloc
        (sizeof(dnode))) == NULL)
    {
        printf("Out of memory !₩n);
        return -1;
     }
    t->key = k;
    tail->prev->next = t;
    t->prev = tail->prev;
    tail->prev = t;
    t->next = tail;
    return k;
}
```

```
int get()
{
    dnode *t;
    int k;
    t = head->next;
    if(t == tail)
    {
        printf("Queue underflow₩n");
        return -1;
    }
    k = t->key;
    head->next = t->next;
    t->next->prev = head;
    free(t);
    return k;
}
```

# Linked list-based Queue (3/4)

- **Clear your queue**
  - **Free all the memory spaces using pointers**

```
void clear_queue()
{
    dnode *t, *s;
    t = head->next;

    while(t != tail)
    {
        s = t;
        t = t->next;
        free(s);
    }

    head->next = tail;
    tail->prev = head;
}
```

```
void print_queue()
{
    dnode *t;
    t = head->next;
    while(t != tail) {
        printf("%-6d", t->key);
        t = t->next;
    }
}
```

➔ **Almost same as the stack operation**

# Linked list-based Queue (4/4)

KU KONKUK UNIVERSITY

## ▪ Implementation of linked list-based queue

- **Definition of the node**

```
dnode *head, *tail;

void main()
{
    int k;
    init_queue();

    Make your code

}
```

**To do list :**

1. Put 3, 6, 9, 1, 6, 3 and print front→rear
2. Conduct get and print front→rear
3. Put 4, 8, 7, 2, 0 (what happens ?)
4. Initialize your queue
5. Conduct get (what happens ?)

☀ **What is the difference with array-based queue ?**

# Summary

- **Applications of linked lists**
  - **Stack and Queue (array-based vs. linked list-based)**
- **Think about the applications for queue**
  - **Think about bank : first-in / first-processed !**
  - **Keyboard buffer, mouse operation, …**